

Fly to Past

第 17 組

B09303109 經濟四曾予忱

B09303063 經濟四謝尚晉

B09303056 財金四陳潤翔

B09303051 經濟四陳泰全

B10703049 財金三柯宥圻

2023 年 12 月 19 日

GitHub 專案連結、展示影片連結

1 系統分析

航空業一直是現代社會的重要一環，它不僅讓人們在地球上輕鬆穿梭，也推動了經濟的全球化發展。在這個快節奏的環境中，航空公司必須不斷適應變化，以確保高效運營並提供最佳的服務。

S.M Walson 曾經說過，將資訊和情報擺第一，錢就會隨之而來。為了應對這些挑戰，我們設計了一套系統幫助需要資料的內部人員提取資訊。而我們這個系統主要是為了方便航空公司內部管理而設計，能夠用於一些營運資料的查詢及修改。我們的資料庫儲存的資料包含航線、航班、飛機、員工，以及顧客和機票的資料等。預期無論是航班調度、機隊管理、員工排班，還是客戶服務資源等資訊，都可以透過這個資料庫管理系統查詢。

根據不同功能及權限，我們管理系統的用戶有幾種身分，分別是 user 以及 admin。user 可以透過該系統來查詢未來需要維修保養的飛機零部件，也就是保養計畫，也可以在維修完零部件後將保養紀錄上傳給系統。如果查詢保養計畫，系統透過儲存在資料庫中的維修時間紀錄來篩選一周內即將需要維修的零部件，讓 user 知道接下來的工作是什麼；如果是上傳保養紀錄，可以在介面中回傳維修部件的部件的維修資訊。user 也能獲取公司的營運狀況資訊，他能夠透過我們的系統搜尋機票的銷售情況。關於這方面，user 可以搜尋航班賣出的座位佔該航班可供銷售的座位比例。搜尋時，user 可以透過選擇想查看的時間段以及起降機場，來衡量不同時間或空間的績效，人事主管也可以透過員工查詢的功能來進行人事管理。admin 的權限除了能使用 user 的功能以外，還能夠新增航班、刪除航班、新增機組人員、刪除機組人員以及更新機票狀況。新增機票這項功能我們透過選取起降時間、機場，並輸入飛機編號、載重以及最大販賣機票數量的方式來達到目的，並透過新增機組人員功能分配職務給員工，如果狀況有變，admin 也可以透過刪除機組人員來進行人員變更、修改機票狀態或者刪除現有

航班來取消航班。

1.1 系統功能

1.1.1 關於商業模式的假設

我們的目標是設計一個管理系統，所以會以管理為出發點，例如我們沒有設計訂票以及其延伸的功能給顧客使用。為了簡化問題，我們的計畫假設服務的站點數量以及航線不會增加，並且每個站點間都會有一個航線連結。每個航班為一個機場到另一個機場，因此一趟需要轉機的旅程中，在我們的系統裡就會算成兩張票。一趟航班有可能會在一個中途的機場加油，這並不會另外算成是兩趟航班，此外商業上一台飛機中途只會加一次油。

機票方面，我們的艙位被區分成經濟艙、商務艙以及頭等艙，並會在資料庫裡用 E、B、F 做紀錄。一張機票可能會有多種狀況，例如乘客反悔取消機票、班機取消之類狀況。我們假設每張機票在購買的同時也成功付款，此時機票的狀況為 P，如果此張機票在購買後被取消了，admin 可以將其狀態更新成 PC，並在退款後更新成 PCR。飛機有可因為外在因素而取消，通常在這種時候航空公司對機票的處理與自行取消的處理方式不同，故我們用 PD 來代表航班取消或變更的機票狀態，並用 PDR 來代表已經成功退款的該類機票。

1.1.2 給 User 的功能

我們希望提供一個能夠查詢機票的銷售情形的功能，以用作航空公司的管理階層加開航班的參考。首先，User 可以選定日期區段、起飛和降落的機場 (也可以不指定)，然後查詢符合條件的航班。對於每一個航班，除了顯示一些基本資訊，還會根據機票的記錄，計算出航班的機票銷售比例。此外，User 也可以選擇用航班編號來查詢指定航班的機票銷售情形。

飛機的部件需要定期進行保養，並且每種部件的保養頻率有所不同，因此安排每天的保養日程可能是一項繁瑣的工作。我們希望提供一個功能讓 User (通常會是維修人員) 能夠查詢未來七天的每一天有哪些部件需要保養。以下說明這個功能運作的機制：首先，對於每個飛機部件，我們查詢其最近一次保養記錄和保養記錄，然後計算出下一次需要保養的日期；最後網頁會呈現出統整後未來七天的保養日程。

此外，系統也提供讓 User 新增保養記錄的功能，以增加功能的完整性。維修人員可以透過網頁表單來新增保養記錄到資料庫，每筆記錄會包含部件編號、保養日期、負責人員和保養成本。我們暫時不考慮保養記錄漏登，以及部件沒有在需要保養的日

期前被保養的情況，以便簡化我們的系統。

user 也可以查詢特定航班的任務被派遣誰，所以我們的系統也有提供相對應的功能。

最後一個 user 的功能是調整機票的狀態，我們假設航空公司可能會遇到客戶打電話給客服取消機票的狀況，此時客服就能使用 user 的功能來進行機票狀態的調整。

1.1.3 給 Admin 的功能

Admin 能夠執行的功能是新增航班，網頁是表單的形式，Admin 能夠輸入航班的必要資訊，接著 Admin 要選擇參與此航班的人員。為了實現這個功能，會需要先查詢所有的機場名稱，以便 Admin 選擇出發和抵達的機場，在表單送出後，需要依照出發和抵達的機場查詢對應的航線編號，最後整理成符合資料表格式的資料儲存到資料庫；另外，Admin 可以派遣員工執行特定航班的勤務，也可以刪除員工來進行人員更動。

Admin 也能夠執行航班的刪除，並且在航班刪除後，此航班的機票會被更改到出發時間最近的航班。同時這些機票的狀態會被標記，以用作未來若需辦理退款時的參考。

我們也提供 admin 員工查詢的功能，分成使用 ID 進行搜尋以及使用職位別，使用 ID 搜尋員工可以得到特定員工的資料，使用職位別搜尋可以得到特定職位的資料。

2 系統設計

2.1 ER Diagram

圖 1 是我們的系統的 ER Diagram，在這個 ERD 中共有 8 個實體 (entity)，分別是 AIRPORT、ROUTE、AIRPLANE、FLIGHT、EMPLOYEE、CUSTOMER、TICKET、PARTS，以及 10 個關係 (relationship)，包括 PURCHASE、IS_OF、DEPART_AT、ARRIVE_AT、BELONG_TO、REFUEL_AT、OWN、EXECUTE、PROCESS、MAINTAIN。AIRPORT 代表航空公司有開設航線站點的機場，ROUTE 代表航線。我們用 PARTS 來記錄飛機可能用到的部件，再用 Own 來記錄飛機擁有部件。MAINTAIN 則代表保養紀錄；REFUEL_AT 用來記錄飛機中途加油。每當有客戶，也就是 CUSTOMER 進行購票時，就會用 TICKET 來記錄，TICKET 裡面紀錄了航班、顧客資訊、時間、票等、座位、價格以及該機票的狀態。員工的崗位和其在飛機的機組人員上的名字可能會不一樣，例如機師在 EXECUTE 這張表中可能會是機長或者副機長。

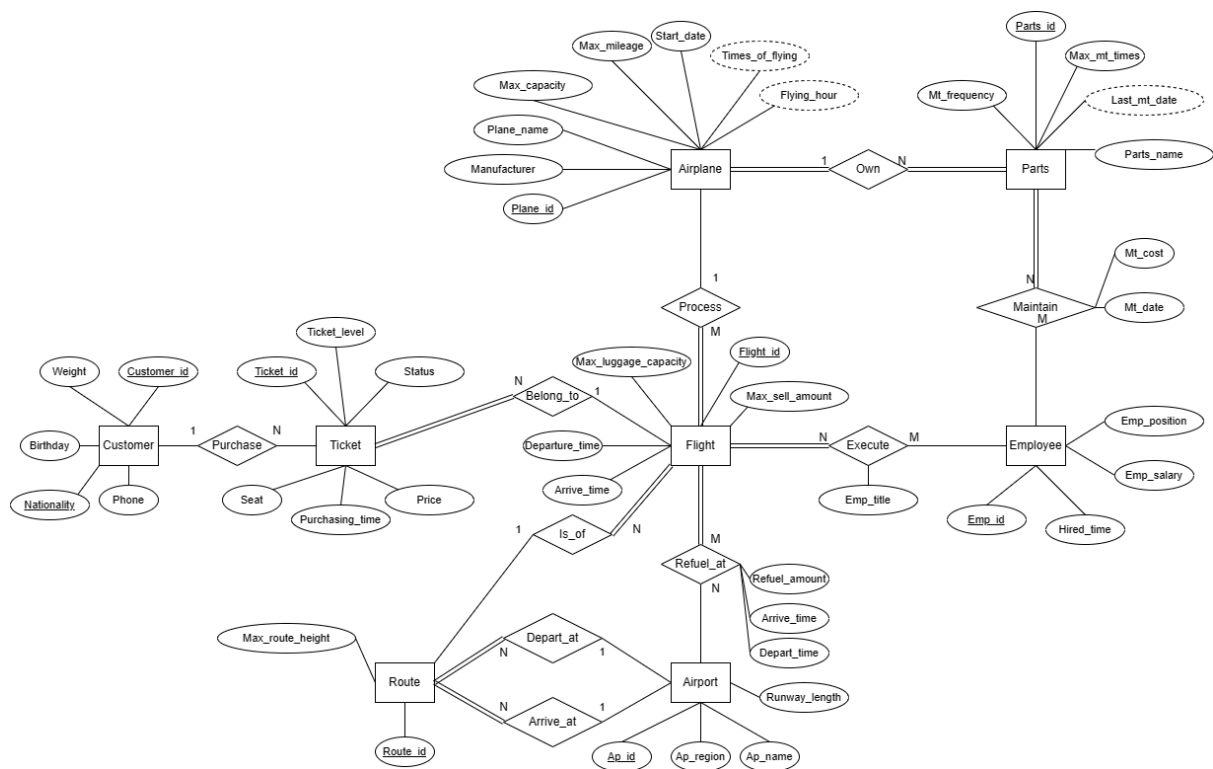


圖 1: ER Diagram

2.2 Relational Database Schema Diagram

我們可以將圖 1 的 ER Diagram 轉換成圖 2 的 Database Schema Diagram，一共有 11 個關聯 (relation)，分別是 AIRPORT、ROUTE、AIRPLANE、FLIGHT、EMPLOYEE、CUSTOMER、TICKET、PARTS、REFUEL_AT、EXECUTE、MAINTAIN。

AIRPORT 的主鍵 (Primary key, PK) 是 Ap_id。ROUTE 的 PK 是 Route_id，因為需要知道起降的機場，所以 Depart_ap 和 Arrive_ap 的外鍵 (Foreign key, FK) 參考 AIRPORT 的 Ap_id。

飛機 AIRPLANE 的 PK 是 Plane_id。航班 FLIGHT 的 PK 是 Flight_id，同時 FK 包括參考到 AIRPLANE 主鍵的 Plane_id 以及參考到 ROUTE 主鍵的 Route_id。FLIGHT_REFUELING 為班機中途經過起降機場以外的機場加油的關聯，所以會由 Flight_id 以及 Visit_ap_id 共為主鍵，他們分別有參考到 FLIGHT 的主鍵 Flight_id 的外鍵以及 AIRPORT 的主鍵 Ap_id 的外鍵，EMPLOYEE 的主鍵是 Emp_id。FLIGHT_CREW 為 FLIGHT 和 EMPLOYEE 之間的關聯，故其主鍵為參考 FLIGHT 的 Flight_id 和 EMPLOYEE 的 Emp_id。

TICKET 的主鍵是 Ticket_id，並且外鍵 Booker_name 和 Booker_id 分別參考到 CUSTOMER 的主鍵 Nationality 和 Id。

MAINTAIN則是為由PARTS和EMPLOYEE的多對多關聯產生的，其主鍵Parts_id和Worker_id分別參考的Parts_id和Emp_id。

2.3 Data Dictionary

我們的資料表共有圖 2 所示的 12 個，各個資料表的欄位相關資訊依序呈現在表 1 到表 11。

Column Name	Meaning	Data Type	Key	Constraint	Domain
Ap_id	機場編號	bigint	PK	Not Null	
Ap_name	機場名稱	varchar (50)		Not Null	
Ap_region	所在國家	varchar (30)		Not Null	
Runway_length	跑道長度	int		Not Null	

表 1: 資料表 AIRPORT 的欄位資訊

Column Name	Meaning	Data Type	Key	Constraint	Domain
Route_id	航線編號	bigint	PK	Not Null	
Depart_ap	起飛機場編號	bigint	FK: AIRPORT(Ap_id)	Not Null	
Arrive_ap	降落機場編號	bigint	FK: AIRPORT(Ap_id)	Not Null	
Max_route_height	最大飛行高度	int		Not Null	
Referential triggers		On Delete	On Update		
Depart_ap: AIRPORT(Ap_id)		Cascade	Cascade		
Arrive_ap: AIRPORT(Ap_id)		Cascade	Cascade		

表 2: 資料表 ROUTE 的欄位資訊

Column Name	Meaning	Data Type	Key	Constraint	Domain
Plane_id	飛機編號	bigint	PK	Not Null	
Plane_name	飛機型號	varchar(10)		Not Null	
Manufacturer	製造商	varchar(50)			
Max_capacity	最大載客人數	int		Not Null	
Max_mileage	最大里程數	int		Not Null	
Start_date	啟用日	date		Not Null	

表 3: 資料表 AIRPLANE 的欄位資訊

Column Name	Meaning	Data Type	Key	Constraint	Domain
Flight_id	航班編號	bigint	PK	Not Null	
Plane_id	飛機編號	bigint	FK: AIRPLANE(Plane_id)	Not Null	
Route_id	航線編號	bigint	FK:ROUTE(Route_id)	Not Null	
Max_luggage_capacity	行李重量上限	int			
Max_sell_amount	最大機票販賣數量	int		Not Null	
Depart_time	起飛時間	datetime		Not Null	
Arrive_time	抵達時間	datetime		Not Null	
Referential triggers		On Delete	On Update		
Plane_id: AIRPLANE(Plane_id)		Cascade	Cascade		
Route_id: ROUTE(Route_id)		Cascade	Cascade		

表 4: 資料表 FLIGHT 的欄位資訊

Column Name	Meaning	Data Type	Key	Constraint	Domain
Flight_id	航班編號	bigint	PK, FK: FLIGHT(Flight_id)	Not Null	
Visit_ap_id	停留機場	bigint	PF, FK: AIRPORT(Ap_id)	Not Null	
Refuel_amount	燃油補充量	int		Not Null	
Arrive_time	降落時間	datetime		Not Null	
Depart_time	再度起飛時間	datetime		Not Null	
Referential triggers		On Delete	On Update		
Flight_id: FLIGHT(Flight_id)		Cascade	Cascade		
Visit_ap_id: AIRPORT(Ap_id)		Cascade	Cascade		

表 5: 資料表 REFUEL_AT 的欄位資訊

Column Name	Meaning	Data Type	Key	Constraint	Domain
Emp_id	員工編號	bigint	PK	Not Null	
Emp_position	員工崗位	varchar (30)		Not Null	
Emp_salary	員工薪水	int		Not Null	
Hired_time	入職日期	date		Not Null	

表 6: 資料表 EMPLOYEE 的欄位資訊

Column Name	Meaning	Data Type	Key	Constraint	Domain
Flight_id	航班編號	bigint	PK, FK: FLIGHT(Flight_id)	Not Null	
Emp_id	員工編號	bigint	PK,FK: AIRPORT(Ap_id)	Not Null	
Emp_title	在此任務擔任職位	varchar(30)		Not Null	{Captain, First Officer, Flight Attendant, Ground Crew, Engineer}
Referential triggers		On Delete	On Update		
Flight_id: FLIGHT(Flight_id)		Cascade	Cascade		
Emp_id: EMPLOYEE(Emp_id)		Cascade	Cascade		

表 7: 資料表 EXECUTE 的欄位資訊

Column Name	Meaning	Data Type	Key	Constraint	Domain
Nationality	國籍	varchar(50)	PK	Not Null	
Id	身分證字號	varchar(20)	PK	Not Null	
Birthday	生日	date			
Weight	體重	demical(4,1)			
Phone	手機號碼	varchar(20)			

表 8: 資料表 CUSTOMER 的欄位資訊

Column Name	Meaning	Data Type	Key	Constraint	Domain
Ticket_id	機票編號	bigint	PK	Not Null	
Booker_nationality	訂票人國籍	varchar(50)	FK: CUSTOMER(Nationality)	Not Null	
Booker_id	訂票人身分證字號	varchar(20)	FK: CUSTOMER(Id)	Not Null	
Flight_id	航班編號	bigint	FK: FLIGHT(Flight_id)	Not Null	
Purchasing_time	購買時間	datetime		Not Null	
Ticket_level	艙位等級	char(1)		Not Null	{E, B, F}
Ticket_Seat	機內座位	char(3)		Not Null	
Status	狀態	varchar(5)		Not Null	{P, PC, PCR, PD, PDR}
Referential triggers		On Delete	On Update		
Flight_id: FLIGHT(Flight_id)		Set Null	Cascade		
Booker_nationality: CUSTOMER(Nationality)		Cascade	Cascade		
Booker_id: CUSTOMER(Id)		Cascade	Cascade		

表 9: 資料表 TICKET 的欄位資訊

Column Name	Meaning	Data Type	Key	Constraint	Domain
Parts_id	部件編號	bigint	PK	Not Null	
Parts_name	部件名稱	varchar(20)		Not Null	
Plane_id	所屬飛機的編號	bigint	FK: AIRPLANE(Plane_id)	Not Null	
Mt_frequency	保養頻率 (天)	int		Not Null	
Max_mt_times	保養次數上限	int		Not Null	
Referential triggers		On Delete	On Update		
Plane_id: AIRPLANE(Plane_id)		Cascade	Cascade		

表 10: 資料表 PARTS 的欄位資訊

Column Name	Meaning	Data Type	Key	Constraint	Domain
Parts_id	部件編號	bigint	PK, FK: PARTS(Parts_id)	Not Null	
Worker_id	負責員工編號	bigint	PK, FK: EMPLOYEE(Emp_id)	Not Null	
Mt_cost	保養花費	int		Not Null	
Mt_date	保養日期	date		Not Null	
Referential triggers		On Delete	On Update		
Parts_id: PARTS(parts_id)		Cascade	Cascade		
Worker_id: EMPLOYEE(Emp_id)		Set Null	Cascade		

表 11: 資料表 MAINTAIN 的欄位資訊

2.4 正規化分析

當設計關聯式資料庫時，我們可以檢視資料庫綱目（database schema）是否滿足正規化（normalization）條件，因此我們將依序從第一正規式（1NF）到第四正規式（4NF）

來說明「航空公司管理系統」的關聯是如何滿足這些規則。

首先，在 1NF 方面，如果每個關聯的屬性都是 simple 且 single-valued，換句話說，在關聯中沒有任何一個屬性是 composite 或 multi-valued，則滿足 1NF。在第 2.2 節中，我們可以看到我們的 schema 中沒有 multi-valued 的狀況，因此我們的 schema 因此滿足 1NF。

在 2NF 方面，如果關聯中的所有非鍵屬性（non-prime attribute）都完全功能相依（fully functional dependency）於任一候選鍵（candidate key），也就是沒有出現部分功能相依性（partial functional dependency），且此關聯滿足 1NF，則滿足 2NF。在第 2.2 節中，我們有將這樣性質的資料分離，導致一張資料表中非主鍵的那些 attribute，都會跟主鍵相關聯，而不會有部分相依。

在 3NF 方面，如果一個關聯中的非鍵屬性都沒有遞移相依（transitively dependency）於主鍵，則滿足 3NF。最後是 4NF，由於此資料庫的所有關聯都不存在多值相依（multi-valued dependency），因此滿足 4NF。同樣我們可以用 2.2 來觀察，我們皆已經在設計過程中，避免這一些錯誤，因此經過分析後，決定此資料庫符合 1NF 至 4NF 之間的所有規則。

3 系統實作

3.1 資料庫建置方式及資料來源說明

在建置一個航空公司資料庫的過程中，本組最難以避免的問題就是無法獲得航空公司的第一手資料，但本組仍盡力從我們對航空業的深入的了解和蒐集的資訊中，用 python 還原出真實感強且能夠進行更多效能測試的數據。我們數據建置的方法中，如名字或生日等數據皆使用 Python 的 Faker 庫生成。其中，飛機資料中的製造商和型號，使用了更多的公司和型號，不拘泥於空客和波音的原因是我們希望測試數據庫的效能和反映機隊的多樣性。員工資料中，員工的職位和在航班上的職務相對應，例如飛行員可以擔任機長或副機長，工程師可以擔任地面維修人員或航空工程師。顧客資料中的國家名稱排除了有標點符號的情況，以確保資料完整性。票務資料中記錄了不同的座次和狀態，用以模仿航空公司即時的票務系統。零件資料包括了不同飛機的詳細零件信息。最後，我們將多個 entity 之間的 relationship 連結起來並且使之符合真實世界的樣貌後，該資料庫便可以成為我們分析或模擬航空行業運營的有效工具。

3.2 重要功能及對應的 SQL 指令

我們將在第 3.2.1 和第 3.2.2 小節之間介紹第 1.1 節裡給 user 以及 admin 的功能，在附帶特定情境下列出完成這些功能所使用之一到多個不等的 SQL 指令。

3.2.1 給 User 的功能

1. 查詢近期需保養的零件：假設我們利用網頁後端得到發送請求當下的日期是「start_date」，並計算出查詢日期的最後一天「end_date」，則查詢對應的指令如下。

```
SELECT pt.parts_id, pt.plane_id, mt.last_mt + pt.
    maintain_frequency AS mt_needed
FROM (
    SELECT parts_id, MAX(maintain_date) AS last_mt
    FROM maintain
    GROUP BY parts_id
) AS mt
JOIN parts AS pt ON mt.parts_id = pt.parts_id
WHERE mt.last_mt + pt.maintain_frequency BETWEEN '{
    start_date}' AND '{end_date}'
ORDER BY mt_needed, pt.plane_id;
```

Listing 1: 查詢需保養的零件 SQL 指令

2. 新增保養記錄：假設 User 要新增一筆保養記錄，其中部件編號是「parts_id」，負責人員的員工編號是「worker_id」，保養日期是「mt_date」，保養成本是「mt_cost」，則查詢對應的指令如下。

```
Insert Into maintain (parts_id , employee_id,
    maintain_date , maintain_fee)
Values ({parts_id}, {worker_id}, '{mt_date}', {mt_cost});
```

Listing 2: 新增保養記錄 SQL 指令

3. 查詢航班和機票銷售情形：讓使用者選擇時間以及查詢機場，機場的部分由後端請求資料庫得來。query 中 User 選定的日期區段是由「start_date」到「end_date」，且出發和抵達的機場編號分別是「departure」和「destination」，則

查詢對應的指令如下方所示，至於「機票販售比例」則會直接在前端處理查詢結果後得到。

```
Select f.*, r.depart_airport, r.arrive_airport, s.  
    ticket_sold  
From (Select *  
      From flight  
      Where DATE(depart_time) Between '{start_date}' And  
            '{end_date}') As f  
Join route As r on f.route_id = r.route_id  
Left Join (Select flight_id , Count(*) As ticket_sold  
          From ticket  
          Group by flight_id) As s on f.flight_id = s.flight_id  
Where r.depart_airport = {departure} And r.arrive_airport  
      = {destination}  
Order By f.depart_time;
```

Listing 3: 查詢航班 SQL 指令

4. 變更機票狀態：若機票狀態需要進行變動，例如將機票狀態變更為已退款狀態，user 可以透過在介面輸入機票的編號以及選擇狀態來進行變更。SQL 指令如下。

```
Update ticket  
Set status = '{status}'  
Where ticket_id = {id};
```

Listing 4: 刪除航班 SQL 指令

5. 查詢特定航班之機組人員：透過在操作介面輸入航班編號「id」，user 可以搜索到該航班的所有機組人員。查詢對應的指令如下。

```
Select *  
From execute  
Where flight_id = {id}  
Order by employee_title;
```

Listing 5: 查詢航班機組人員 SQL 指令

3.2.2 給 Admin 的功能

1. 新增航班：admin 想要新增航班時，首先在操作介面中選定起降機場，我們會預先從資料庫找出所有機場供其選擇。然後要選擇調派的飛機，同樣會先從資料庫找出飛機的編號。再來起飛降落日期時間時間、輸入最大載重以及最大機票販售數量，實作的 SQL 指令如下。

```
INSERT INTO flight (  
    flight_id,  
    plane_id,  
    route_id,  
    depart_time,  
    arrive_time,  
    max_luggage_capacity,  
    max_sell_amount  
) VALUES (  
    {id},  
    {airplane},  
    {route_id},  
    '{depart_time}',  
    '{arrive_time}',  
    {max_luggage_capacity},  
    {max_sell_amount}  
);
```

Listing 6: 新增航班 SQL 指令

2. 刪除航班：若 admin 需要刪除取消的航班，可以在介面中輸入要取消的航班編號「id」來實現，此時後端會發送刪除航班的指令以及修改該航班之機票，SQL 指令如下。

```
Delete From flight  
Where flight_id = {id};  
  
Update Ticket  
Set Status = PC
```

```
Where Flight_id = {id};
```

Listing 7: 刪除航班 SQL 指令

3. 新增機組人員：新增的航班尚未派遣員工執行，故 admin 需要安排員工執行，admin 需要輸入航班編號「flight_id」、「employee_id」和「employee_title」。SQL 指令如下。

```
Insert into execute (  
    flight_id, employee_id, employee_title  
)  
Values (  
    {flight_id}, {employee_id}, '{employee_title}'  
);
```

Listing 8: 新增機組人員 SQL 指令

4. 刪除機組人員：若人員有變動，可以透過輸入航班編號「flight_id」以及機組人員編號「employee_id」進行刪除。SQL 指令如下。

```
Delete From execute  
Where flight_id = {flight_id}  
And employee_id = {employee_id};
```

Listing 9: 刪除機組人員 SQL 指令

5. 查詢特定職務之員工資料：先由後端查詢所有職務以及該職務的人數後讓 admin 選定「position」，前端會顯示所有該職務員工之資料。查詢對應的指令如下。

```
select *  
from employee  
where employee_position = '{position}';
```

Listing 10: 查詢航班 SQL 指令

6. 查詢特定員工第二種方法為直接透過輸入員工編號「id」查找員工資料。查詢對應的指令如下。

```

select *
from employee
where employee_id = {id};

```

Listing 11: 查詢航班 SQL 指令

3.3 SQL 指令效能優化與索引建立分析

由於該系統的目的之一是需要實現旅客快速查詢航班和其起飛的機場和降落的機場，因此為flight的depart_date和ticket的flight_id做兩個索引，語法如下。

```

CREATE INDEX idx_flight_dt
ON flight(depart_time);

CREATE INDEX idx_ticket_flight
ON ticket(flight_id);

```

Listing 12: 建立索引 SQL 指令

用以下的查詢進行測試。

```

Select f.*, r.depart_airport, r.arrive_airport, s.ticket_sold
From (Select *
      From flight
      Where DATE(depart_time) Between '2015-11-20'
      And
      '2023-11-26') As f
Join route As r on f.route_id = r.route_id
Left Join (Select flight_id, Count(*) As ticket_sold
          From ticket
          Group by flight_id) As s on f.
          flight_id = s.flight_id
Where r.depart_airport = 48867996 And r.arrive_airport =
      68976744
Order By f.depart_time;

```

Listing 13: 測試用指令

建立索引前運行五次的結果分別是 0.121 秒、0.130 秒、0.099 秒、0.114 秒、0.139 秒，平均運行時間約為 0.120 秒，標準差約為 0.015 秒；建立索引後運行五次的結果分別是 0.077 秒、0.089 秒、0.073 秒、0.071 秒、0.070 秒，平均運行時間約為 0.076 秒，標準差約為 0.008 秒。建立索引後平均運行時間的降低和標準差的減小都表明索引查詢性能產生了積極的影響，因為使用flight_id和route_id的索引可以加快資料表之間連結操作的速度，使資料庫能夠更快地匹配相對應的資料。

因應工程師在處理大量維護記錄時所需做的查詢，即在maintain表找出每個零件的最後維護日期，做一個索引，使得對於每個parts_id，其相關的最後維護日期可以更快地被確定，加快聯接操作的處理速度。語法如下。

```
CREATE INDEX idx_maintenance_record_mt_date
ON maintain ( maintain_date );
```

Listing 14: 建立索引 SQL 指令

用以下的查詢進行測試。

```
Select mt.parts_id , mt.last_mt + pt.maintain_frequency As
    mt_needed
From (Select parts_id , Max(maintain_date) As last_mt
      From maintain
      Group by parts_id) As mt
Join parts As pt on mt.parts_id = pt.parts_id
Where mt.last_mt + pt.maintain_frequency Between '2020-11-20'
      And '2023-11-26'
Order By mt_needed;
```

Listing 15: 測試用指令

建立索引前運行五次的結果分別是 0.107 秒、0.103 秒、0.111 秒、0.114 秒、0.101 秒，平均運行時間約為 0.107 秒，標準差約為 0.005 秒；建立索引後運行五次的結果分別是 0.064 秒、0.090 秒、0.073 秒、0.068 秒、0.077 秒，平均運行時間約為 0.074 秒，標準差約為 0.010 秒。建立索引後平均運行時間降低了。因為查詢結果是根據下次維護所需的日期進行排序。因此索引可以更加迅速地處理這些數據被排序的數據進行尋找。

3.4 Transaction Management and Concurrency Control

我們的功能大致上不會因為 Concurrency 而產生問題。然而，以新增航班為例，我們會先查詢所有航班的編號以產生新的編號，這時 Concurrency 可能導致新的航班編號互相衝突，因此其中一個 Transaction 會成功提交，其餘則會無法成功 Insert。我們希望未來能更完善例外處理這方面。

4 分工資訊

在本次專案中，我們的分工資訊如下：

曾予忱：後端環境和資料庫連接，以及撰寫程式碼和部分指令

謝尚晉：文字報告的部份以及前端網頁的撰寫

陳泰全：細節處理，部分書面報告，Demo 影片錄製

柯宥圻：簡報製作以及錄製展示影片

陳潤翔：資料庫建置與管理

5 組員心得

1. 曾予忱：我在專案負責的是後端環境建置和連接資料庫，以及撰寫部分 SQL 指令。雖然很多功能具備基本雛形，但在細節和例外處理還有待完善。這次的經驗讓我稍微了解後端運作，以及操作資料庫時的必要處理。
2. 謝尚晉：在這一次的專案中，我的部分是文字報告的部份以及前端網頁的撰寫，由於是第一次撰寫前端，只寫出了一些簡單的靜態網頁，深刻的認知到還有許多不足，例如可以用一個 CSS 檔來解決排版一致性的問題，對於 script 用法的理解還需要花更多時間研究。
3. 陳泰全：我在這次專案中做的事情比較雜，首先我了解如何用專業的方式去呈現成果，其重要性不亞於做好報告，以及幫助有困難的同學解決問題，令我在幫助同學的同時也可以學習資料庫專案相關的技能。
4. 柯宥圻：在這一次的專案中，我主要被分配的工作是文字紙本的部分，設計發想，簡報製作以及最後的影片繳交，而沒有真的有投入寫 code 的部分，雖然會覺得自己在這樣的編排架構之下沒有實際上做到甚麼事情，但是實際上了解隊友們所提出的 code，並且融會貫通好好講出自己該說的結果，試圖讓其他人知道

我們做的事情是有意義且具有架構、有被優化過的，我想這也是一種訓練，最後我愛小傑。

5. 陳潤翔：在這一次的專案中，我的部分是資料蒐集和資料庫建構，因為資料蒐集的部分難以從真實的航空公司處取得，因此資料主要是由 python 生成，但是由此生成多種關聯性和合理性問題都很嚴重，因此讓我了解到資料的取得不易。

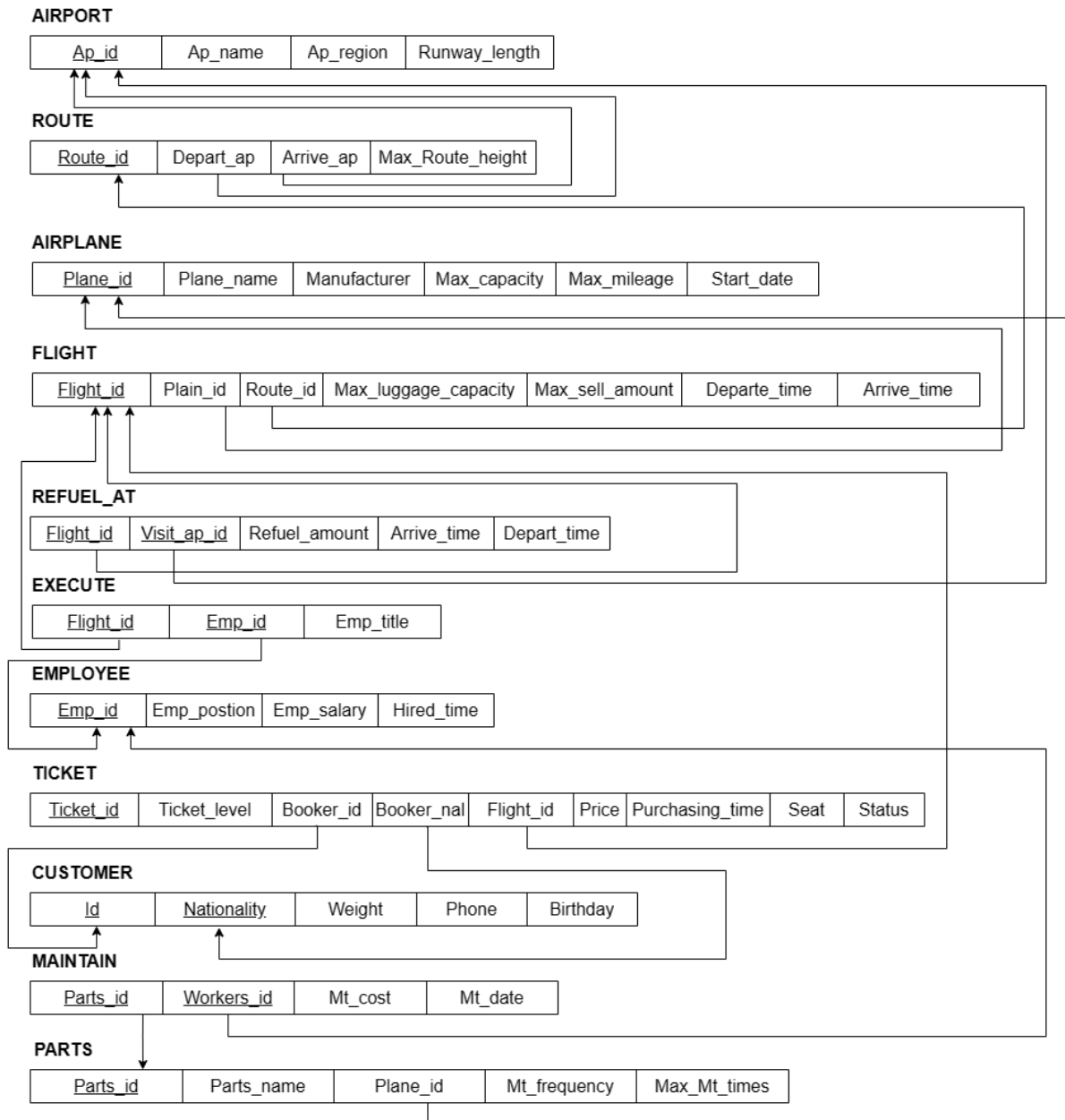


圖 2: 「Fly to Past」的 Relational Database Schema Diagram